Milestone 6, Part 1: Hand Isomorphism and the Rules of the Full Game

holdem-cfr walkthrough

Where milestone 6 is going

Milestones 4 and 5 solve a *subgame*: one board, two given ranges, a fixed bet menu. A bot that plays full heads-up no-limit from the first card has to deal with two problems the subgame solver never sees.

The first is the preflop and flop. The full game is far too large to solve exactly, so the bot needs a coarse **blueprint** strategy for the whole game. Like Libratus and Pluribus, it gets one by abstracting both the cards (similar hands share a bucket) and the bets (a small menu of sizes), then training with Monte Carlo CFR.

The second is the ranges. When the bot reaches the turn in a real hand, it needs ranges to feed the subgame solver. Those come from the blueprint by Bayes' rule: given what each player did, which hands would they have played that way?

This part builds the two foundations everything else stands on. One is an exact way to number hands so that equivalent hands share a number. The other is the betting rules of the real game.

Why number hands at all

Card abstraction assigns every situation (hole cards plus board) to a bucket, and the assignment has to be stored somewhere. The natural store is an array indexed by situation. The raw number of situations is enormous: 1,326 hole-card pairs times C(50, 5) = 2,118,760 river boards is 2.8 billion. But most of them are copies of each other under suit relabelling. A♠K♠ on Q♠7♥2♦ plays exactly like A♥K♥ on Q♥7♠2♦: swap spades and hearts and one becomes the other. Count only distinct classes and the river has 123,156,254. That is about 23 times fewer, small enough that a one-byte bucket per class is a 123 MB table.

What we need is a **perfect hash**: a function that maps each situation to an index in 0..N, gives equivalent situations the same index, gives different classes different indices, and leaves no gaps. Waugh (2013) gives one. The construction below is the same idea, derived from scratch.

The construction, one layer at a time

A suit's signature. Fix one suit, say spades, and write down which spade ranks appear in each round. With the board treated as one set, that means which spade ranks are in the hole cards and which are on the board. Call this the suit's *signature*. The four suits each have one. A suit relabelling only changes *which suit carries which signature*, so the class of a situation is exactly the **multiset** of the four signatures. Everything else is turning that multiset into an integer.

Layer 1: ranking a signature. A suit's *shape* is how many of its cards land in each round, for example one hole card and two board cards. Within a shape, a signature is a choice of ranks for the hole round, then a choice from the *remaining* ranks for the board round. Each choice is a subset of a fixed size, and subsets of fixed size have a classic perfect ranking, the **colex rank**: for positions $p_1 < p_2 < \ldots < p_k$, the rank is $C(p_1, 1) + C(p_2, 2) + \ldots + C(p_k, k)$. This is the same combinatorial number system that indexed the 1,326 two-card combos in milestone 4.

For the second round, first *compress* the mask by deleting the ranks already used, so the subset lives in $13 - c$ positions. Combining the two rounds with a mixed radix gives a dense rank for the signature within its shape.

Layer 2: the multiset. Sort the four shapes. The sorted list is the situation's *configuration*. Suits with the same shape are interchangeable, so their signatures form a multiset. A multiset of k values chosen from N possibilities is a *combination with repetition*, and those have their own colex rank: sort $x_0 \le x_1 \le \ldots$ and sum $C(x_i + i, i + 1)$. There are $C(N + k - 1, k)$ of them. Groups with different shapes combine with another mixed radix.

Layer 3: configurations. Enumerate every configuration once, at startup, by splitting each round's cards over four suits in every possible way, sorting, and deduplicating. Lay the configurations end to end, each at its own offset. The index of a situation is its configuration's offset plus its rank inside that configuration.

Each layer is a bijection onto its range, so the composition is a bijection onto 0..N. In code, a situation's index costs a few popcounts, a few binomial lookups and one hash-map lookup for the configuration.

A mistake the tests caught immediately

The first version used four rounds (hole, flop, turn, river) and the unit test expected the textbook numbers 169, 1,286,792, 13,960,050 and 123,156,254. Preflop and flop matched, but the turn and river came out as 55,190,538 and 2,428,287,420. That looks like a bug and is not one. A quick lower bound settles it. The raw (hole, flop, turn) count is 1,326 × 19,600 × 47 = 1.22 billion, and dividing by the 24 suit permutations leaves at least 50.9 million classes, so 13,960,050 *cannot* be the count when the turn card is kept separate from the flop.

The two sets of numbers answer different questions. Kept as separate rounds, the board remembers the order the cards were dealt: a perfect-recall index. With the board as one set, a heart turn on a flop with two hearts is the same as a heart flop card plus another turn card. That is the *imperfect-recall* view card abstraction uses, because hand strength does not care when a card arrived. The indexer now takes the round sizes as a parameter, and the tests pin all six published numbers:

	preflop	**flop**	**turn**	**river**
deal order remembered (2, 3, 1, 1)	169	1,286,792	55,190,538	2,428,287,420
board as one set (2, n)	169	1,286,792	13,960,050	123,156,254

Proving it is a bijection

Matching the sizes shows the index space is the right size, not that the map is right. So the gate compares the indexer against **brute-force canonicalization**: try all 24 suit permutations, sort the hole cards and the board, and keep the smallest encoding. Two situations are isomorphic exactly when their canonical forms are equal.

- **Preflop, exhaustively:** all 1,326 hands produce exactly 169 indices, each with one canonical form. - **Flop, exhaustively:** all 25,989,600 (hole, flop) situations, in parallel. Every index in 0..1,286,792 is hit, every index has exactly one canonical form, and every canonical form has exactly one index. That is a complete proof for the flop indexer. - **Turn and river, by sampling:** 400,000 random situations per street. Each is relabelled by a random suit permutation and has its hole and board cards shuffled, and the index must not change. Across all samples, index equality and canonical-form equality must always coincide. The test also checks that enough samples collided for this to mean something.

The rules of the full game

`rules.rs` implements heads-up no-limit as played in the Annual Computer Poker Competition. The defaults are 100 big blinds deep with 50/100 blinds. The button posts the small blind, acts first preflop and last afterwards. A raise is a "raise to" amount, the player's total contribution, so the history prints in ACPC notation like `r300c/cr600c/`.

The details that trip people up are tested one by one:

- A limp gives the big blind the option to check or raise. - The minimum raise is the previous raise *increment*: after a raise to 300 over a 100 blind, the minimum re-raise

is to 500. After 900, it is 1,500. - The minimum bet is one big blind. - A short stack may go all-in for less than a minimum raise. - After an all-in and a call, the board runs out with no more betting. - Payoffs are correct for folds, showdowns and split pots.

A property test plays 500 random hands with random legal actions and random stack depths. It requires that:

- a legal action always exists; - contributions never shrink and never exceed the stack; - every hand ends within 60 actions; - every payoff sums to zero.

Next

Part 2 computes card buckets. For every river class, it needs hand strength against a uniform opponent range, using the sorted-rank trick from milestone 4 board by board. For turn and flop classes, it needs $E[HS^2]$, the average over future cards of squared strength, which rewards drawing hands for their potential. Each class is then assigned to a percentile bucket per street, stored in arrays indexed by exactly the numbers built here.